

Modules 8–9: Streamlit & Data Apps › Neural Networks & Deep Learning

PROGRAM	ASSESSMENT CODE	TOTAL TIME	STACK
Data Science	M9-A1	4 Hours	Python 3

GENERAL INSTRUCTIONS

Attempt all sections. Write your code in Python 3 unless stated otherwise. For Streamlit tasks, submit a screenshot of your running app alongside the source file. For deep learning tasks, submit your .py or .ipynb file along with a screenshot of training output and evaluation metrics. Label every task clearly with its section and task number. All work must be your own.

A Section A — Concept Application

Answer all questions. Each question presents a real-world scenario followed by a concept-specific question. Questions 1–3 draw on Module 8 (Streamlit); Questions 4–6 draw on Module 9 (Neural Networks & Deep Learning).

SCENARIO

S
1

You are building an order-tracking dashboard for a food delivery startup's operations team. The dashboard needs a city dropdown to filter results, a slider to set a delivery-time range, and a sidebar so analysts can navigate between an Overview page and a Restaurant Performance page without cluttering the main canvas.

Question: Which specific Streamlit components would you use for each of these three requirements, and how does placing navigation controls in the sidebar — rather than on the main page — improve usability when the dashboard has multiple filter options?

SCENARIO

S
2

You are developing a live delivery-performance analytics app for a food aggregator. The product manager wants an interactive line chart showing hourly order counts that analysts can zoom into, hover over for exact values, and toggle between restaurants — features that `st.line_chart` does not support out of the box.

Question: Why would you use Plotly over Streamlit's built-in `st.line_chart` for this requirement, and what are the two essential lines of code needed to render a Plotly figure object inside a Streamlit app?

SCENARIO

S
3

You are deploying a food-delivery-time prediction model for a logistics company. The trained scikit-learn model is saved as a .pkl file, and the Streamlit app must let dispatchers enter order details and receive a predicted delivery time instantly without the model reloading on every interaction.

Question: Describe the sequence of steps — from saving the model file to displaying the prediction — that makes this Streamlit prediction app work correctly, and explain why wrapping the model load function with `st.cache_resource` is critical for production performance.

SCENARIO

S
4

You are building a neural network to classify incoming food orders into three urgency tiers — Standard, Priority, and Express — based on order value, distance from restaurant, and estimated preparation time. Your current model uses a sigmoid activation on the output layer.

Question: Which activation function should the output layer use for this three-class classification task, and why does using sigmoid on the output layer produce incorrect probability distributions when you need mutually exclusive class probabilities across all three tiers?

SCENARIO

S
5

You are training an ANN on a food delivery platform's dataset to predict peak-hour demand. After 30 epochs the training accuracy reaches 96% but validation accuracy stalls at 71%, indicating severe overfitting. Your model has three Dense hidden layers.

Question: Compare Dropout regularization and L2 weight regularization as strategies to reduce this gap. For a dense multi-layer neural network, which would you apply first in practice, at what typical rate or strength, and what different mechanism does each use to discourage overfitting?

SCENARIO

S
6

You are designing a system for a food delivery platform to automatically classify customer-submitted food photos as 'acceptable quality' or 'poor quality'. A colleague suggests using a simple ANN with pixel values flattened into a 1D input vector. You are considering a CNN instead.

Question: Explain the key architectural difference between an ANN and a CNN that makes CNNs significantly more effective for image classification, and describe what a convolutional filter learns from spatial data that fully connected layers cannot efficiently capture.

B

Section B — Practical Coding Tasks

Complete all tasks. Tasks 1–2 draw on Module 8 (Streamlit); Task 3 draws on Module 9 (Neural Networks & Deep Learning); Task 4 is integrative across both modules. Follow the requirements listed under each task.

Task 1: Delivery Summary Dashboard

Build a Streamlit app that reads an uploaded food delivery CSV file and displays interactive business metrics with city-level filtering. (Foundational — single Streamlit data flow: upload › filter › display.)

- Requirement 1 — Add a file uploader widget that accepts a CSV file; if no file is uploaded, display a `st.info()` placeholder message instructing the user to upload delivery data.
- Requirement 2 — Display total orders, average delivery time (in minutes), and total revenue as three side-by-side metric cards using `st.metric()` with appropriate labels.
- Requirement 3 — Add a sidebar selectbox listing all unique cities found in the uploaded data; update all three metric cards to reflect only the selected city's records.
- Requirement 4 — Render a bar chart below the metric cards showing order count per restaurant for the selected city using `st.bar_chart()`.

Task 2: Delivery Time Prediction UI

Build a Streamlit app that loads a pre-trained scikit-learn regression model and lets dispatchers predict estimated delivery time by entering order details. (Applied — model inference integrated into a Streamlit form with input validation.)

- Requirement 1 — Load a saved model file named `delivery_model.pkl` using `joblib`, wrapped inside a function decorated with `st.cache_resource` so the model is loaded only once per session.
- Requirement 2 — Create input widgets for: distance (slider, 1–50 km), order value in Rs (number input), and time of day (selectbox with options: Morning, Afternoon, Evening, Night).
- Requirement 3 — On a Predict Delivery Time button click, encode the time-of-day selection as an integer (0–3), assemble the feature array, pass it to the loaded model, and display the predicted delivery time in minutes using `st.success()`.
- Requirement 4 — Handle any `ValueError` or model file load failure gracefully using a `try-except` block, displaying a descriptive `st.error()` message without crashing the app.

Task 3: Order Demand Classification Network

Build, train, and evaluate a Keras ANN to classify food delivery demand into Low, Medium, and High categories from time-based and environmental features. (Intermediate — neural network design combined with multi-class classification output.)

- Requirement 1 – Generate a synthetic dataset of at least 500 samples with three numeric features – hour_of_day (0–23), day_of_week (0–6), and temperature_celsius (10–45) – and a three-class target label; apply one-hot encoding to the target using to_categorical().
- Requirement 2 – Build a Sequential model with two Dense hidden layers (64 and 32 units respectively) using ReLU activation, a Dropout layer with rate 0.3 after each hidden layer, and a final Dense output layer with Softmax activation.
- Requirement 3 – Compile with categorical_crossentropy loss and the Adam optimizer; train for 30 epochs with a 0.2 validation split and store the History object.
- Requirement 4 – Print the final test accuracy after evaluating on a held-out test set, and plot training vs validation accuracy across all epochs using Matplotlib with clearly labelled axes and a legend.

Task 4: Streamlit Deep Learning Inference App

Integrate the trained Keras demand classification model from Task 3 into a Streamlit interface, combining Module 8 UI patterns with Module 9 neural network inference into a single deployable app. (Integrative – Streamlit + Keras + Plotly across both modules.)

- Requirement 1 – Save the trained model from Task 3 using `model.save('demand_model.keras')` and load it in a new Streamlit app using `tf.keras.models.load_model()`, wrapped in `st.cache_resource`.
- Requirement 2 – Add input widgets for hour of day (slider, 0–23), day of week (selectbox, Monday–Sunday encoded as 0–6), and temperature in °C (number input, range 10–45).
- Requirement 3 – On a Predict Demand button click, assemble and reshape the input into a (1, 3) NumPy array, pass it through the loaded model, and display the predicted demand class (Low / Medium / High) with the associated confidence percentage using `st.success()` or `st.warning()` depending on the class.
- Requirement 4 – Display a Plotly bar chart showing the raw probability scores for all three demand classes (Low, Medium, High) so dispatchers can visually assess model confidence alongside the final prediction.

C

Section C – Mini Capstone Project

Mini Project: Food Delivery Intelligence Dashboard

Objective:

Build a multi-page Streamlit web application that serves as an end-to-end food delivery intelligence platform – combining interactive data exploration, real-time deep learning inference, and model transparency – consolidating Streamlit UI design, CSV handling, data visualization, and Keras neural network deployment from Modules 8 and 9 into a single production-ready app.

Your project must:

- The app must be structured with at least four sidebar-navigable pages: Home (platform overview), Data Explorer (upload & visualise delivery data), Demand Predictor (live ANN inference), and Model Info (architecture and training history).
- The Data Explorer page must accept a CSV upload, display summary statistics (total orders, average delivery time, revenue breakdown), and render at least two interactive Plotly charts — one showing hourly order volume trends and one showing delivery time distribution by restaurant.
- The Demand Predictor page must load a pre-trained Keras ANN (saved from Section B) using `st.cache_resource`, accept user inputs for hour, day of week, and temperature, and output the predicted demand class with a Plotly bar chart of per-class confidence scores.
- The Model Info page must display the loaded model's architecture summary (layer names, output shapes, and total parameters) as a formatted table and render training accuracy and loss history as labelled Plotly line charts.
- The app must handle all edge cases gracefully — missing file uploads, out-of-range inputs, and model load failures — using `st.error()` or `st.warning()`, and must include a `requirements.txt` and a one-paragraph README describing how to run the app locally.

D

Section D — AI-Augmented Learning

STEP 1 • BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a program that:

- Requirement 1 — Builds a Keras Sequential neural network to classify food delivery orders into three priority levels (Low, Medium, High) using at least three numeric input features.
- Requirement 2 — Trains the model for 20 epochs and prints training accuracy after every epoch.
- Requirement 3 — Plots training and validation loss curves side by side using Matplotlib.
- Requirement 4 — Saves the final trained model to a file named `food_priority_model.keras` and reloads it to make and print one test prediction.

STEP 2 • TEST & DEBUG (WITHOUT AI)

Then, working without AI, test the code and find at least one bug, limitation, or improvement in the AI's solution. Fix it yourself. Common mistakes to look for: using `sigmoid` instead of `softmax` on the output layer for a multi-class problem, forgetting to one-hot encode the target labels, using `mean_squared_error` instead of `categorical_crossentropy` as the loss function, or omitting a `validation_split` during training.

SUBMIT

All 3 items required

- 1 The exact prompt(s) you gave the AI tool.
- 2 The AI's original code and your corrected version, clearly labelled.
- 3 A 3–4 line note explaining what you changed and why the AI's version needed the correction.